Trabajo Práctico · Unidad 7

Food Store en MongoDB: pedidos y detalle · Base de Datos II · Tecnicatura Universitaria en Programación · UTN

Alumno: Moran, Joaquín Leandro · Trabajo individual.
Repositorio: github.com/moranjoaquin487-lgtm/tpi-food-store, carpeta `trabajos-practicos/unidad7_nosql`.
Motores: PostgreSQL 18 (fuente de verdad) y MongoDB con `mongosh`, en mi PC (servidor standalone).

Consigna	Archivo
Estado del pedido (faltaba en el esquema)	`01_estado_pedido.sql`
1–2 · Modelo y migración (`insertMany`)	`02_exportar_pedidos.sql` → `pedidos_export.json` → `cargar_mongodb.js`
3 · Reglas de integridad y qué se embebió	`cargar_mongodb.js` (control de R1 y validador `$jsonSchema`), sección 1
4 · Consultas a–d en MongoDB	`consultas_mongodb.js`
5 · Equivalentes en PostgreSQL y verificación	`consultas_postgresql.sql`, `verificar_consultas.sh` → `evidencias/tabla_verificacion.md`
6–7 · Comparación con IA y contraste	`prompts/comparacion_ia.md`, `docs/respuestas_ia/comparacion.md`, `pruebas_afirmaciones_ia.js`, `pruebas_integridad_mongodb.js`

1 · Modelo documental y migración

Origen en PostgreSQL

Tabla	Clave primaria	Claves foráneas	Qué aporta al documento
`pedido`	`id_pedido`	`id_cliente` → `cliente`	Fecha, estado, forma de pago
`detalle_pedido`	`id_detalle` (y `UNIQUE (id_pedido, id_producto)`)	`id_pedido` → `pedido`, `id_producto` → `producto`	Cantidad y precio histórico de cada línea
`cliente`	`id_cliente`	—	Nombre y apellido
`producto` / `categoria`	`id_producto` / `id_categoria`	`id_categoria` → `categoria`	Nombre del producto y de la categoría

El esquema no tenía estado de pedido. Se agregó la columna `estado` (`PENDIENTE`, `CONFIRMADO`, `ENTREGADO`, `CANCELADO`) en `01_estado_pedido.sql`, porque la consigna trabaja con él.

Colección `foodstore.pedidos`

```
{
  _id: 1,                    ← id_pedido
  fecha: ISODate("…"),
  estado: "ENTREGADO",
  forma_pago: "EFECTIVO",
  cliente: {                 ← embebido
    id_cliente: 1,
    nombre: "Ana",
    apellido: "Gutiérrez"
  },
  lineas: [                  ← embebido
    { id_detalle: 1,
      id_producto: 1,
      producto: "Pan de campo 1kg",
      categoria: "Panificados",
      cantidad: 2,
      precio_unitario: Decimal128("2200.00"),
      subtotal: Decimal128("4400.00") },
    …                        (4 líneas en total)
  ],
  total: Decimal128("6851.78"),   ← al migrar
  origen: { motor: "postgresql",
            tablas: [ … 5 tablas ] }
}
```

Identificadores conservados: `_id` = `id_pedido`, `cliente.id_cliente`, `lineas[].id_detalle` y `lineas[].id_producto`. Cada documento se puede rastrear hasta su fila de origen.

Migración:

1. `02_exportar_pedidos.sql` arma el JSON en PostgreSQL. El subconjunto son los pedidos con al menos un producto del catálogo semilla: **62 pedidos**, con todas sus líneas.
2. `cargar_mongodb.js` los carga con `insertMany()`: 62 leídos, **62 insertados**.
3. Los importes viajan como texto y se guardan como `Decimal128`, para no perder centavos en un `double`.

Reglas de integridad que se conservan

- **R1. El total es la suma de las líneas, al centavo.** Se controla en la carga, con aritmética entera en centavos. Si un solo documento no cumple, no se carga ninguno.
- **R2. Toda línea tiene cantidad entera mayor que 0 y precio mayor o igual que 0, y todo pedido tiene al menos una línea.** La hace cumplir el validador `$jsonSchema` de la colección, también en las inserciones futuras. La prueba con cantidad 0 fue rechazada (*Document failed validation*).

Qué se embebió y qué no

Se embebieron las **líneas**: solo existen dentro de su pedido (es una composición), siempre se leen junto con él y su precio es el **histórico de la venta**, que no tiene que cambiar si después cambia el precio del producto. Ese es justamente el caso donde copiar el dato es correcto. Del **cliente** se embebieron solo el id, el nombre y el apellido, como foto del momento de la venta: el mail y el teléfono no hacen falta para mostrar un pedido. **No** se embebieron el **stock** ni el **precio actual** del producto: cambian con cada venta y los comparten todos los pedidos, así que copiarlos obligaría a actualizar muchos documentos por cada cambio. Siguen en PostgreSQL.

2 · Consultas y verificación

Consulta	Filtro en MongoDB	Resultado
a) Todos los pedidos	`find({}, {_id: 1, estado: 1})`	62 pedidos
b) Por estado	`find({estado: "CANCELADO"})`	10 pedidos
c) Con un producto	`find({"lineas.producto": "Yerba mate 1kg"})`, con proyección `$elemMatch`	9 pedidos
d) Total superior al umbral	`find({total: {$gt: umbral}})`	21 pedidos sobre $ 10 928,97

Umbral de d): el ticket promedio del subconjunto migrado, calculado con `$avg` y redondeado a centavos. Separa los pedidos más grandes que lo habitual y se recalcula si cambian los datos, en lugar de ser un número elegido a ojo. PostgreSQL usa el mismo criterio: `ROUND(AVG(total), 2)`.

Tabla de verificación (`verificar_consultas.sh`)

Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	¿Coinciden los IDs?
b) Estado `CANCELADO`	10	10	**Sí**
c) Producto «Yerba mate 1kg»	9	9	**Sí**

IDs de b) en los dos motores: 3, 2083, 30413, 52083, 80413, 102083, 108743, 152083, 180413, 260013. IDs de c): 2, 2083, 48092, 52083, 98092, 102083, 144101, 152083, 198092.

Diferencias: ninguna. Coincidieron las cantidades y los IDs, y también la consulta d): 21 pedidos sobre $ 10 928,97 en los dos motores. La diferencia que puede aparecer es la **deriva**: la vista `v_pedidos_migrados_mongo` se recalcula en cada consulta, así que un pedido nuevo con un producto semilla, creado después de exportar, lo cuenta PostgreSQL y no MongoDB. La corrección es exportar de nuevo, o filtrar en PostgreSQL por los IDs que se exportaron.

3 · Evaluación de la tecnología e IA

Prompt (`prompts/comparacion_ia.md`): describe el modelo actual, el acceso frecuente a pedidos completos y la coherencia de cantidades, precios y stock, y pide comparar PostgreSQL y MongoDB. Lo respondió OpenCode (agente Plan, modelo Big Pickle, en mi PC); la respuesta completa está en `docs/respuestas_ia/comparacion.md`. Recomendó mantener PostgreSQL como fuente de verdad y, a lo sumo, MongoDB como copia de lectura. Antes de responder abrió `informe_u7.md`, que ya tenía el borrador de la conclusión, y lo cita: queda declarado.

Contraste de dos afirmaciones

Afirmación de la IA	Contraste	¿Se sostiene?
«Posible con transacciones multi-documento (requiere replica set)»	`pruebas_afirmaciones_ia.js`: mi servidor es standalone y la transacción falló con *This MongoDB deployment does not support retryable writes*. La documentación de transacciones también pide replica set o cluster: mongodb.com/docs/manual/core/transactions/	**Sí.** Para una venta atómica en MongoDB habría que montar un replica set, aunque sea de un nodo: otro costo operativo a favor de PostgreSQL.
« `$jsonSchema` solo valida tipo/rango por campo; no puede expresar la suma. Queda a cargo de la aplicación»	La prueba 2 de `pruebas_integridad_mongodb.js` confirma la primera parte: `$jsonSchema` aceptó un total de 1.00. Pero un validador con `$expr`, que compara `total` con la suma de `cantidad × precio_unitario`, aceptó el pedido real (4 líneas, $ 6851,78) y **rechazó** el de total 1.00. Es una función documentada: mongodb.com/docs/manual/core/schema-validation/specify-query-expression-rules/	**A medias.** Sobre `$jsonSchema` es cierto, pero la conclusión es falsa: el motor puede hacer cumplir R1 con `$expr`. Lo que sigue sin tener MongoDB es integridad referencial entre colecciones y un control del stock que comparten todos los pedidos.

Laboratorio (`evidencias/pruebas_integridad.txt`): MongoDB aceptó una línea con un `id_producto` inexistente (no hay claves foráneas); `$jsonSchema` aceptó un total que no es la suma de las líneas; rechazó la cantidad 0; 200 lecturas del pedido completo con `findOne` tardaron 240 ms en total.

4 · Conclusión

Conviene **mantener PostgreSQL** como fuente de verdad de los pedidos, por dos criterios:

1. **Coherencia transaccional.** Registrar una venta toca el pedido, sus líneas y el stock de varios productos en una sola transacción, con triggers que impiden vender sin stock. En MongoDB, el stock quedaría en otra colección: habría que usar transacciones de varios documentos y reprogramar en la aplicación los controles que hoy hace el motor. El laboratorio mostró además que MongoDB acepta productos inexistentes y totales incoherentes si la aplicación no los controla.
2. **Patrón de acceso.** Leer un pedido completo es más simple en un documento, con una sola operación y sin joins. Pero en Food Store ese acceso ya se resuelve con índices sobre `id_pedido`, y el volumen no justifica un segundo motor que habría que sincronizar.

Si en el futuro hiciera falta una vista documental, por ejemplo un historial de pedidos para la aplicación, la opción más económica es combinar: PostgreSQL como fuente y MongoDB, o una columna `JSONB` en el propio PostgreSQL, como copia de solo lectura regenerable.

Declaración de uso de IA (DUIA)

Herramientas	**Claude** (Anthropic): escribió los scripts (estado del pedido, exportación, carga, consultas, verificación y pruebas), el borrador del informe, incluida la conclusión, y este documento. **OpenCode** (agente Plan, en mi PC): respondió la comparación de la consigna 6.
Qué se aceptó y qué no	De OpenCode se tomaron dos afirmaciones y se contrastaron en mi MongoDB: una se sostiene y la otra solo a medias (sección 3).
Verificación	Corrí la carga, las consultas, la verificación contra PostgreSQL y las pruebas en mi PC; las salidas están en `evidencias/`. Los pedidos y clientes son datos de prueba.